جامعة حفر الباطن
University of Hafr Al Batin

10th May, 2024

Computer Science Engineer

Project (CSE 202): Library Record System

Submitted By

Student Name	**Student ID**
Fay Naif Alharbi	2221003431
Danah Muhammad Al-Anazi	2221001396
Aliayh Saud Al-Mutairi	2221005295
Reema Yahya Al Faifi	2211005551
Lamya Faisal Alahmari	2221005417

Table of Contents

Summary 1

Introduction 3

Analysis Stage 5

System Requirements 5

Design Stage 6

Data structures 6

User Interface (Output Sample) 7

Implementation Stage 8

Coding 8

Testing 16

Conclusions 18

Recommendations 20

References 22

Summary

This Java project provides a comprehensive solution for managing a library's book records using the Binary Search Tree (BST) data structure. The system's efficiency and effectiveness are highlighted by its ability to handle various operations such as adding, searching, removing, and displaying books.

Key Features:

- Robust Book Representation: Each book is encapsulated in a BookNode object, which stores key attributes including the book's unique ID, title, author, rating, and availability status. This object-oriented approach ensures a robust and cohesive representation of a book.
- Efficient Data Structure: The system uses a BST for storing book records. This data structure is renowned for its efficiency in handling search and insertion operations. In this system, the book's rating serves as the key, ensuring that all books are stored in an order governed by their ratings.
- Powerful Search Mechanism: The searchBookById(int bookID) function provides a powerful tool for locating specific books based on their unique IDs. This function leverages the properties of the BST to provide efficient and fast search capabilities.
- Comprehensive Display of Books: The library's entire collection of books can be displayed in ascending order of their ratings. This is made possible by performing an inorder traversal of the BST, which outputs the books in sorted order.
- Book Removal Capability: The system also allows for the removal of books from the library. The removeBook(int bookID) method enables users to delete a book from the library using its unique ID. This enhances the system's functionality and adaptability,

accommodating changes in the library's collection over time.

The system is interactive, with a console-based menu driving the user interface. The users can easily choose from a list of operations, making the system user-friendly and straightforward to navigate. This project showcases the effective use of Java and BST in creating a simple yet efficient library record system.

Introduction

The ability to manage data effectively is a cornerstone of modern software applications. As the complexity and volume of data that these applications handle continue to skyrocket, it is vital to have a systematic way to organize, store, and retrieve data. This is where data structures come into play, serving as the foundation for data management and significantly influencing the overall performance and efficiency of software applications.

Numerous types of data structures exist, each with its unique characteristics and specific use cases. Familiar data structures such as arrays, linked lists, stacks, queues, trees, and graphs are frequently used across a range of applications. Each of these data structures has its own merits and limitations, making it essential for developers to understand their functionalities to choose the most suitable one for their tasks.

In this particular project, our focus shifts to the implementation of a Library Record System that demands efficient handling of book data. To cater to this need, we have opted to use a Binary Search Tree (BST) data structure. Known for their excellence in maintaining sorted data and offering fast search capabilities, BSTs are an ideal choice for managing book records based on their ratings. This facilitates users in quickly locating books within specific rating parameters, ensuring that the library's collection remains methodically organized and easily navigable.

In conclusion, through strategic use of data structures, we are able to create an organized, user-friendly, and efficient Library Record System that simplifies the management of book records, delivering a seamless user experience.

Analysis Stage

System Requirements

The Library Record System aims to provide a user-friendly and efficient way to manage book records within a library setting. Here are the specific requirements for the system:

Functionality:

- Add Book:
  - Allow librarians to add new book records to the system.
  - Capture essential information such as book ID , title, author, and rating.
  - Automatically assign the book as "available" upon adding.
- Search Book by ID:
  - Enable librarians to search for specific books using their unique IDs.
  - Efficiently locate the book record within the system and display its details (title, author, rating, availability).
- Deleting a book:
  - allows the user to delete the book
  - if the book no longer exists
- Display Books:
  - Provide an option to display all book records in the system.
  - Present the books in ascending order based on their ratings.
  - Include details such as bookID, title, author, rating, and availability status for each book.

Design Stage

Data structures

The Library Record System primarily relies on a Binary Search Tree (BST) to efficiently manage and organize book data. Here's a breakdown of the data structures involved:

1. BookNode Class:

- This class represents a single node within the BST.
- Each BookNode object stores the following information:
    - bookID (int): A unique identifier for the book.
    - title (String): The title of the book.
    - author (String): The author of the book.
    - available (boolean): Indicates whether the book is currently available for borrowing (true) or borrowed (false).
    - rate (float): The rating of the book.
    - left (BookNode): A reference to the left child node in the BST (for nodes with lower ratings).
    - right (BookNode): A reference to the right child node in the BST (for nodes with higher ratings).

2. Library Class:

- This class represents the overall library and manages the BST of book records.
- The Library class contains the following key components:
    - root (BookNode): A reference to the root node of the BST.
    - Methods for performing various operations on the BST, including:
    - addBook: Inserts a new BookNode into the BST while maintaining the sorted order

based on ratings.

- searchBook: Searches for a specific book by its ID using the properties of the BST.
- borrowBook: Updates the availability status of a book to "borrowed".
- returnBook: Updates the availability status of a book to "available".
- displayBooks: Traverses the BST in inorder to display all book records in ascending order of ratings.

Benefits of Using a BST:

- Efficient Searching: BSTs enable fast searching for books based on their ratings, achieving an average time complexity of O (log n). This is crucial for quickly locating specific books within a large library collection.
- Sorted Order: Books are automatically maintained in a sorted order based on their ratings, facilitating easy browsing and retrieval of books within specific rating ranges.
- Dynamic Structure: BSTs can efficiently handle insertions and deletions of book records, allowing for a dynamic and evolving library collection.

User Interface (Output Sample)

The user interface presents a menu with options to add books, remove a book, display all books, search for a book, and exit.

When adding a book, the user is prompted to enter details such as bookID, title, author, and rate. They can continue to add books until they choose to stop.

Error handling is straightforward, with simple messages displayed for invalid inputs or actions like trying to remove a book that does not exist in the system

Implementation Stage

Coding

```
import java.util.Scanner;


class BookNode {
   int bookID;
   String title;
   String author;
   boolean available;
   float rate;
   BookNode left, right;


   public BookNode(int bookID, String title, String author, float rate, boolean available) {
      this.bookID = bookID;
      this.title = title;
      this.author = author;
      this.rate = rate;
      this.available = available;
      this.left = null;
      this.right = null;
   }
}
```

```
class Library {
    BookNode root;
    private int nextBookID;
    public Library() {
        root = null;
        nextBookID = 1;
    }

    public void addBook(String title, String author, float rate , boolean available) {
        BookNode newNode = new BookNode(nextBookID, title, author, rate , available);
        nextBookID++;
        if (root == null) {
            root = newNode;
        } else {
            BookNode current = root, parent = null;
            while (true) {
                parent = current;
                if (newNode.rate < current.rate) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
```

```
                current = current.right;
                if (current == null) {
                    parent.right = newNode;
                    return;
                }
            }
        }
    }
}

public void searchBookById(int bookID) {
    BookNode foundBook = searchBookByIdRecursive(root, bookID);
    ;
    if (foundBook != null) {
        System.out.println("Book Found: ID: " + foundBook.bookID + " || Title: " +
foundBook.title +
                " || Author: " + foundBook.author + " || Rate: " + foundBook.rate +
                " || Available: " + (foundBook.available ? "Yes" : "No"));
    } else {
        System.out.println("No book found with ID: " + bookID);
    }
}

private BookNode searchBookByIdRecursive(BookNode node, int bookID) {
```

```
    if (node == null) {
        return null;
    }
    if (bookID == node.bookID) {
        return node;
    }
    BookNode foundNode = searchBookByIdRecursive(node.left, bookID);
    if (foundNode == null) {
        foundNode = searchBookByIdRecursive(node.right, bookID);
    }
    return foundNode;
}

public void displayBooks() {
    displayBooksRecursive(root);
}
public void removeBook(int bookID) {
    BookNode current = root, parent = null;
    while (current != null) {
        if (current.bookID == bookID) {
            if (current == root) {
                root = (current.left!= null)? current.left : current.right;
            } else if (parent.left == current) {
                parent.left = (current.left!= null)? current.left : current.right;
```

```
            } else {
                parent.right = (current.left!= null)? current.left : current.right;
            }
            return;
        } else if (bookID < current.bookID) {
            parent = current;
            current = current.left;
        } else {
            parent = current;
            current = current.right;
        }
    }
}


private void displayBooksRecursive(BookNode node) {
    if (node != null) {
        displayBooksRecursive(node.left);
        System.out.println("ID: " + node.bookID + "|| Title: " + node.title +
            "|| Author: " + node.author + "|| Rate: " + node.rate + "|| Available: " +
node.available);
        displayBooksRecursive(node.right);
    }
}
```

```
}

public class LibraryRecordSystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Library library = new Library();

        library.addBook( "The Hitchhiker's Guide to the Galaxy", "Douglas Adams", 4.2f, true);
        library.addBook( "The Lord of the Rings", "J.R.R. Tolkien", 4.5f, true);
        library.addBook("Pride and Prejudice", "Jane Austen", 3.9f, true);

        while (true) {
            System.out.println("\nChoose an action:");
            System.out.println("1. Add item");
            System.out.println("2. Print list");
            System.out.println("3. Search Item");
            System.out.println("4. Remove item");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            int choice = scanner.nextInt();

            switch (choice) {
                case 1:
                    boolean available = false;
```

```
    System.out.print("Enter Book Title : ");
    scanner.nextLine();
    String title = scanner.nextLine();
    System.out.print("Enter Book Author : ");
    String author = scanner.nextLine();
    System.out.print("Enter Book rate: ");
    float bookRate = scanner.nextFloat();
    System.out.print("Enter Book available: ");
    scanner.nextLine();
    String availableChar = scanner.nextLine();
    if (availableChar.toLowerCase().equals("y")) available = true;
    library.addBook(title, author, bookRate, available);
    break;
case 2:
    library.displayBooks();
    break;
case 3:
    System.out.print("Enter Book ID to search: ");
    library.searchBookById(scanner.nextInt());
    break;
case 4:
    System.out.print("Enter Book Id : ");
    library.removeBook(scanner.nextInt());
    break;
```

```
            case 5:
                System.out.println("Exiting...");
                System.exit(0);
                break;
            default:
                System.out.println("Invalid choice.");
        }
    }
  }
}
```

Testing

```
Choose an action:
1. Add item
2. Print list
3. Search Item
4. Remove item
5. Exit
Enter your choice: 1
Enter Book Title : The Hunger Games
Enter Book Author : Suzanne Collins
Enter Book rate: 4.34
Enter Book available: y
```

Figure 1: Add new Book

```
Choose an action:
1. Add item
2. Print list
3. Search Item
4. Remove item
5. Exit
Enter your choice: 2
ID: 1|| Title: Pride and Prejudice|| Author: Jane Austen|| Rate: 3.9|| Available: true
ID: 2|| Title: The Hitchhiker's Guide to the Galaxy|| Author: Douglas Adams|| Rate: 4.2|| Available: true
ID: 4|| Title: The Hunger Games|| Author: Suzanne Collins|| Rate: 4.34|| Available: true
ID: 3|| Title: The Lord of the Rings|| Author: J.R.R. Tolkien|| Rate: 4.5|| Available: true
```

Figure 2: show all book in system

```
Choose an action:
1. Add item
2. Print list
3. Search Item
4. Remove item
5. Exit
Enter your choice: 3
Enter Book ID to search: 2
Book Found: ID: 2 || Title: The Hitchhiker's Guide to the Galaxy || Author: Douglas Adams || Rate: 4.2 || Available: Yes
```

Figure 3: search a book in system

```
Choose an action:
1. Add item
2. Print list
3. Search Item
4. Remove item
5. Exit
Enter your choice: 4
Enter Book Id : 3

Choose an action:
1. Add item
2. Print list
3. Search Item
4. Remove item
5. Exit
Enter your choice: 2
ID: 1|| Title: Pride and Prejudice|| Author: Jane Austen|| Rate: 3.9|| Available: true
ID: 2|| Title: The Hitchhiker's Guide to the Galaxy|| Author: Douglas Adams|| Rate: 4.2|| Available: true
ID: 4|| Title: The Hunger Games|| Author: Suzanne Collins|| Rate: 4.34|| Available: true
```

Figure 4: remove a book, show books after removed

```
Choose an action:
1. Add item
2. Print list
3. Search Item
4. Remove item
5. Exit
Enter your choice: 5
Exiting...
```

Figure 5: Exit

Conclusions

The Library Record System succinctly exemplifies the transformative power of data structures in managing, structuring, and organizing information within a software application. By leveraging a Binary Search Tree (BST) to store book records, the system ensures efficient search, insertion, and deletion operations, all the while maintaining the books in a meticulously sorted order based on their ratings. This not only facilitates users in swiftly locating specific books but also allows them to browse the library collection in a structured and easily navigable manner.

This project underscores the significance of selecting suitable data structures to tackle specific challenges. The properties of the BST, which include efficient searching and maintenance of sorted data, harmonize perfectly with the requirements of the Library Record System. This alignment provides optimal performance in searching and maintaining order within the collection, enhancing the overall user experience.

Additionally, the system employs automatic ID generation, a feature that significantly simplifies the process of adding new books to the collection. This feature also reinforces data integrity by ensuring that each book is assigned a unique identifier, eliminating the chance of data duplication or confusion.

Moreover, the system's ability to display all books in ascending order of their ratings provides users with a comprehensive view of the library's collection. This, coupled with the system's efficient search and removal operations, makes the Library Record System a robust and user-friendly tool for managing and browsing book records.

In conclusion, this project showcases the potent combination of data structures and Java programming in creating a simple yet effective library record management system. It serves as a testament to the importance of strategic data management and the role it plays in enhancing software application performance and user experience.

Recommendations

In order to further enhance the functionality, reliability, and user-friendliness of the Library Record System, several recommendations are proposed.

Comprehensive Error Handling: Strengthening the error handling capabilities of our system is crucial. This could involve implementing stringent validation checks for user inputs such as book ID, title, author, and rating before a book is added to the library. Additionally, exceptions that may arise during operations like adding, searching, and deleting books should be effectively managed to prevent any disruption in system functioning and to ensure a smooth user experience.

Advanced Sorting and Search Functionality: Incorporating more sophisticated sorting options and search functionality can significantly improve the user experience. Users could be offered the ability to sort books based on various parameters, including title, author, or rating. The search functionality could be expanded to enable users to search for books not only by ID but also by title or author. This will provide a more versatile and comprehensive book search experience.

User Authentication and Authorization: Adding user authentication and authorization mechanisms can ensure that only authorized individuals can add or remove books. This could be achieved by implementing a login system that requires a username and password, as well as a role-based access control system that defines different levels of access based on the user's role.

Improved User Interface and User Experience: Enhancing the user interface for better navigability and user experience can make the system more engaging and easier to use. This could involve designing clearer prompts for user input, providing more informative feedback about the results of operations, and creating a visually appealing layout that enhances the overall user experience.

Automated Testing and Continuous Integration: Implementing automated testing and continuous integration can help identify potential issues early in the development cycle, ensuring the system's robustness and reliability. Test cases that cover a range of scenarios, including potential errors and edge cases, can help maintain the system's resilience and improve its overall quality.

Data Backup and Recovery: Implementing a data backup and recovery mechanism could provide an extra layer of security for the library's data. This would ensure that even in the event of a system failure, the data could be restored without significant loss.

Performance Optimization: Regular analysis of the system's performance metrics can help identify any potential bottlenecks. Based on this analysis, performance optimization strategies could be implemented to ensure the system remains efficient and responsive even as the library's collection grows.

By implementing these recommendations, we can ensure that the Library Record System evolves to meet the growing needs of its users, delivering a robust, efficient, and user-centric experience.

References

[1] Lec 10

[2] https://www.geeksforgeeks.org/tree-data-structure/

[3] https://www.cppforschool.com/project/library-management-system.html

[4] Chat GPT, Gemini , Tabnine